

C++ 문자열 클래스

class == data type

```
char A[] = "Grade";  
char* A = newchar [100];  
int len = strlen(A);  
int n = sizeof(A);
```

- C++ 라이브러리는 string이라는 클래스를 제공
- string class 내부에 문자배열과 이 문자열을 처리할 수 있는 멤버 함수로 이루어짐
- string 클래스는 C 문자열과 구분하기 위해, C++ 문자열 객체라고 부름
- C 문자열은 문자의 배열이며, C 문자열을 처리할 때는 <cstring> 라이브러리에 정의된 함수를 활용해야 함
- C++ 문자열은 클래스 자료형이며, C++ 문자열을 처리할 때는 <string> 라이브러리에 있는 멤버 함수를 활용

```
#include <string>
```

C++ 문자열의 생성과 복사, 길이

생성

```
string strg1;      // Creating an empty string object
string strg2("hello");      // The string "hello"
string strg3 = "hello everybody";
```

복사

```
string src = "foo bar foo bar foo bar";
string desc = "hi everybody!";
desc = src;
```

길이

```
string s1;
string s2 = "123";
string s3 = "abcdefg"; // (dot) : 객체의 멤버 함수에 접근하는 연산자
cout << "s1: " << s1.size() << endl; // 0
cout << "s2: " << s2.size() << endl; // 3
cout << "s3: " << s3.size() << endl; // 7
```

빈 문자열 확인하기

- `empty` 함수는 문자열의 크기가 0이라면 `true`를 리턴하고, 이외의 경우에는 `false`를 리턴

```
bool fact = strg.empty();           // Checking emptiness
```

C++ 문자열의 비교()

연산자를 사용한 비교

- 두 문자열의 연산자를 사용한 비교는 불 자료형(true 또는 false)으로 비교 결과를 리턴
- 연산자를 사용한 비교에서는 서브 문자열 비교를 따로 할 수 없음
- 만약 서브 문자열을 비교하고 싶다면, 서브 문자열을 만든 뒤에 비교해야 함

```
// Comparing two C++ strings  
bool result = strg1 oper strg2;  같으면 true (즉, 1)
```

연산자 : == != < <= > >=

C에서는 strcmp(s1, s2) 를 사용했다
if(strcmp(s1, s2) == 0)
 같으면 0

C++ 문자열의 비교()

Program 10.21 *Using logical operators to compare strings*

```
1  /*****
2  * The program to test logical operators to compare two strings. *
3  *****/
4  #include <string>
5  #include <iostream>
6  using namespace std;
7
8  int main ( )
9  {
10     // Creation of four C++ strings
11     string strg1;
12     string strg2 (5, 'a');
13     string strg3 ("Hello Friends");
14     string strg4 ("Hi People", 4); // Hi P
15     // Using six logical operators (relational and equality)
16     cout << "strg1 < strg2 : " << boolalpha << (strg1 < strg2);
17     cout << endl;
18     cout << "strg2 >= strg3: " << boolalpha << (strg2 >= strg3);
19     cout << endl;
20     cout << "strg1 = strg2: " << boolalpha << (strg1 = strg2);
```

C++ 문자열의 비교()

Program 10.21 *Using logical operators to compare strings*

```
21     cout << endl;  
22     cout << "Hi P != strg4: " << boolalpha << ("Hi P" != strg4);  
23     return 0;  
24 }
```

Run:

```
strg1 < strg2 : true  
strg2 >= strg3: true  
strg2 < strg3: false  
strg1 == strg2: false  
Hi P != strg4: false
```

C++ 문자열의 결합

- 예시에서 temp, temp1, temp2는 모두 문자열 클래스의 객체

```
string strg = temp;           // Assignment 생성  
string strg += temp;         // Compound Assignment  
string strg = temp1 + temp2;  // Addition
```

전체 이름 출력하기()

Program 10.23 *Printing the full name of a person*

```
1  /*****
2  * A program to show concatenation of strings and characters. *
3  *****/
4  #include <string>
5  #include <iostream>
6  using namespace std;
7
8  int main ( )
9  {
10     // Declarations
11     string first, last;
12     char init;
13     // Input first, last, and initial
14     cout << "Enter first name: ";
15     cin >> first;
16     cout << "Enter last name: ";
17     cin >> last;
18     cout << "Enter initial: ";
19     cin >> init;
20     // Printing the full name in one format
```


전체 이름 출력하기()

Program 10.23 *Printing the full name of a person*

```
21     cout << endl;
22     cout << "Full name in first format: ";
23     cout << first + " " + init + "." + " " + last << endl << endl;
24     // Printing the full name in another format
25     cout << "Full name in second format: ";
26     cout << last + ", " + first + " " + init + ".";
27     return 0;
28 }
```

Run:

Enter first name: John

Enter last name: Brown

Enter initial: A

Full name in first format: John A. Brown

Full name in second format: Brown, John A.

C문자열의 C++ string 객체로의 변환과, vice versa

```
char cs1[] = "Are you a string too?";
string cpps;
// c style 문자열 의 string 객체로 변환
cpps = cs1; // 문자열과 여러 함수들도 존재하는 객체가 형성됨
cpps[0] = 'B';
cout << "cpps : " << cpps << endl;

//string 객체에서 c style 문자열 얻기
string cpps2 = "Yes, I am.";
const char* cs2 = NULL;
cs2 = cpps2.c_str(); //문자열의 시작주소를 반환한다. 오직 읽기 용도로만 사용할 수있음

int len = cpps2.size(); //10
char* cs3 = new char[len + 1]; //복사를 할 char배열준비
strcpy_s(cs3, len + 1, cpps2.c_str()); //복사, cs3는 독립적인 c style의 문자열이 됨
Cout << cs3 << endl;
Cs3[0] = 'Z'; // 수정 가능
Cout << cs3 << endl;
Delete[] cs3; cs3 = NULL;
```

C++ 문자열의 입출력()

입출력 연산자

- 지금까지 기본 자료형을 출력할 때 사용했던 연산자들은 입력 객체(istream)와 출력 객체(ostream)가 있는 멤버 함수
- 입력 연산자는 입력 스트림에서 공백을 만나기 전까지 문자를 하나하나 읽어 들임
- 즉 띄어쓰기 또는 줄바꿈 등이 있으면 읽기가 중단
- 출력 연산자는 문자열 전체를 출력 스트림에 출력(write)
- 입력 연산자와 다르게 별도의 제한이 없음

```
cin >> strg;           // inputting characters for a string
cout << strg;          // outputting the character of a string
```

C++ 문자열의 입출력()

Program 10.13 *Using input/output operators*

```
1  /*****
2  * A program to test input/output operators with string objects. *
3  *****/
4  #include <string>
5  #include <iostream>
6  using namespace std;
7
8  int main ( )
9  {
10     // Constructing a default object
11     string strg;
12     // Inputting and outputting values for the strg object
13     cout << "Input the string: " ;
14     cin >> strg;
15     cout << strg << endl;
16     return 0;
17 }
```

C++ 문자열의 입출력()

Program 10.13 *Using input/output operators*

Run:

Input the string: Hello
Hello

Run:

Input the string: Hi my friends
Hi

C++ 문자열의 입출력()

getline 함수

- 입력 연산자는 입력 스트림에서 입력을 읽는 것을 멈추게 만드는 구분 기호가 있음. 만약 다른 구분 기호를 사용해서 입력을 읽는 것을 멈추게 하는 등 추가적인 제어를 하고 싶은 경우에는 istream 객체의 getline 함수를 사용
- 첫 번째 형태는 구분 기호로 '\n'을 사용하므로 한 줄을 읽음. 두 번째 형태는 구분 기호를 사용자가 지정하는 것. 원하는 구분 기호까지 읽어 들이게 만들 수 있음

```
getline(cin, strg); // The input stops with '\n'
```

```
getline(cin, strg, 'c'); // The input stops with  
character c
```

C++ 문자열의 입출력()

Program 10.14 *Using getline for input*

```
1  /*****
2  * A program to test the getline function with strings.      *
3  *****/
4  #include <string>
5  #include <iostream>
6  using namespace std;
7
8  int main ( )
9  {
10     // Constructing a default object
11     string strg;
12     // Creating a string made of a single line
13     cout << "Enter a line of characters: " << endl;
14     getline (cin, strg);
15     cout << strg << endl << endl;
16     // Creating a string made of multiple lines
17     cout << "Enter lines of characters ended with $: " << endl;
18     getline (cin, strg, '$');
19     cout << strg;
20     return 0;
```

C++ 문자열의 입출력()

Program 10.14 *Using getline for input*

```
21 }
```

Run:

Enter a line of characters:

This is a line of text.

This is a line of text.

Enter lines of characters ended with \$:

This is a multi-line set of
characters to be
stored in a string.\$

This is a multi-line set of
characters to be
stored in a string.